

Programming Language Management

Rohit Goswami · 2021-09-19 · shell · lang

`nix` aside¹, I recently shifted to using `asdf` to manage different language versions.

```
git clone https://github.com/asdf-vm/asdf.git ~/.asdf --branch v0.8.1
```

The main reason to prefer `asdf` over competing language specific options like `rvm` or `pyenv` or `nvm` and company is simply uniformity of the interface. This can be coupled with `zinit snippet OMZ::plugins/asdf` for loading the completions. Note that the installation steps can take a while especially if `openssl` is being installed.

Configuration

Actually one weird aspect of `asdf` is that it pollutes `$HOME` by default instead of respectfully storing its configuration in `$HOME/.config/asdf` like every other program. To change this; exporting `ASDF_CONFIG_FILE` works. Essentially a `zsh` configuration with `zinit` would look like this²:

```
if [[ ! -d ~/.asdf ]]; then
  mkdir ~/.asdf
  git clone https://github.com/asdf-vm/asdf.git ~/.asdf
fi

export ASDF_CONFIG_FILE=~/.config/asdf/asdfrc"
zinit snippet OMZ::plugins/asdf
```

`asdf` has an option; `legacy_version_file = yes` which is meant to ensure that existing solutions (e.g. `.nvrc` or `.node-version`) are compatible. There is also `$HOME/.tool-versions` for trickle-down language versions; that is language-version pairs defined in such files are activated for the directory and lower sub-directories. The rest of a reasonable configuration ([described here](#)) might look like:

```
legacy_version_file = yes
always_keep_download = no
```

Ruby

This [plugin](#) leverages `ruby-build` and so can take a pick up an extended set of environment variables; it supports `.ruby-version` files as well.

```
asdf plugin add ruby
asdf list all ruby
asdf install ruby 3.0.2
asdf global ruby 3.0.2 # Convenience
```

NodeJS

This [plugin](#) respects `.nvrc` and `.node-version` files.

```
asdf plugin add nodejs
asdf global nodejs latest
asdf install nodejs latest
```

Python

Builds with [python-build](#).

```
asdf plugin add python
asdf install python 3.9.7
asdf global python 3.9.7
```

Direnv

I'm a huge fan of `direnv`; and a [fantastic little plugin](#) (with neat `hyperfine` benchmarks) for `asdf` removes a layer of shim indirection while playing nicely with `direnv`.

```
asdf plugin-add direnv
asdf install direnv latest
asdf global direnv latest
```

This comes with an associated set of shell profile instructions:

```
# File: ~/.zshrc
# Hook direnv into your shell.
eval "$(asdf exec direnv hook zsh)"
# A shortcut for asdf managed direnv.
direnv() { asdf exec direnv "$@"; }
```

Along with a commiserate `$HOME/.config/direnv/direnvrc` requirement:

```
source "$(asdf direnv hook asdf)"
```

Now every new `.envrc` can start with `use asdf` which will now speed up evaluations.

Conclusions

This method appears to be more robust than remembering the various idiosyncrasies and logic of a host of other tools.